

Acceptance behavior theories and models in software engineering — A mapping study

Jürgen Börstler^{a,*}, Nauman bin Ali^a, Kai Petersen^{a,b}, Emelie Engström^c

^a Blekinge Institute of Technology, Karlskrona, Sweden

^b University of Applied Sciences, Flensburg, Germany

^c Lund University, Lund, Sweden

ARTICLE INFO

Dataset link: <https://doi.org/10.5281/zenodo.8060722>

Keywords:

Acceptance behavior
Technology adoption
Theory use in software engineering
TAM
TPB
TRA
Fitness
Innovation diffusion

ABSTRACT

Context: The adoption or acceptance of new technologies or ways of working in software development activities is a recurrent topic in the software engineering literature. The topic has, therefore, been empirically investigated extensively. It is, however, unclear which theoretical frames of reference are used in this research to explain acceptance behaviors.

Objective: In this study, we explore how major theories and models of acceptance behavior have been used in the software engineering literature to empirically investigate acceptance behavior.

Method: We conduct a systematic mapping study of empirical studies using acceptance behavior theories in software engineering.

Results: We identified 47 primary studies covering 56 theory uses. The theories were categorized into six groups. Technology acceptance models (TAM and its extensions) were used in 29 of the 47 primary studies, innovation theories in 10, and the theories of planned behavior/ reasoned action (TPB/TRA) in six. All other theories were used in at most two of the primary studies. The usage and operationalization of the theories were, in many cases, inconsistent with the underlying theories. Furthermore, we identified 77 constructs used by these studies of which many lack clear definitions.

Conclusions: Our results show that software engineering researchers are aware of some of the leading theories and models of acceptance behavior, which indicates an attempt to have more theoretical foundations. However, we identified issues related to theory usage that make it difficult to aggregate and synthesize results across studies. We propose mitigation actions that encourage the consistent use of theories and emphasize the measurement of key constructs.

1. Introduction

Since its inception, software engineering research aimed to impact practice [1]. However, the research often falls short of its goal [2, 3]. Several researchers have empirically investigated success factors for software process improvement initiatives [4–6], technology transfer [7], and industry-academia collaboration models [8,9] to identify factors likely to facilitate the adoption of technologies or ways of working. Such empirical work, however, often lacks a theoretical frame of reference.

In software engineering, the relation between empirical inquiry and theory is well recognized [10]. Having a theoretical frame of reference informs what research questions to pose and supports the analysis and interpretation of the data [11]. It increases the credibility and validity of empirical research and enables generalization beyond specific contexts [12].

In other fields, e.g., the health sciences and applied psychology [13], specialized theories and models exist and are used to explain or predict whether a change in current behavior and practice will likely be adopted. Theoretical foundations can rapidly advance our understanding of acceptance behavior in software engineering and improve the accuracy of our predictions of acceptance or adoption of a technology or way of working.

Theories of acceptance behavior explain or predict changes in human behavior beyond the assumed rational decision-making [14]. They will enable an understanding of the factors that influence software engineers' acceptance of changes to their ways of working, which will support software development organizations in adopting new software development practices and technologies.

Fig. 1 shows a high-level overview of the concepts of acceptance behavior theories. Behavioral intention is the central construct of acceptance behavior theories and is used as a proxy for actual behavior [15]. The main predictors for behavioral intention are influenced

* Corresponding author.

E-mail address: jurgen.borstler@bth.se (J. Börstler).

<https://doi.org/10.1016/j.infsof.2024.107469>

Received 26 June 2023; Received in revised form 4 April 2024; Accepted 6 April 2024

Available online 16 April 2024

0950-5849/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

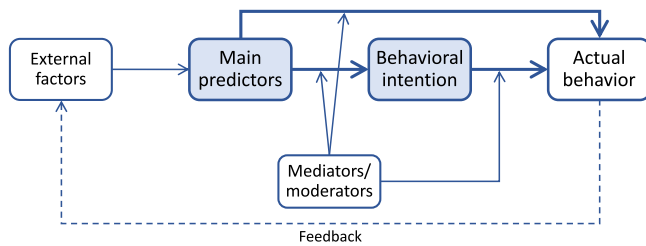


Fig. 1. Basic concepts of acceptance behavior models and theories, slightly adapted from [14] (grey boxes = core constructs).

by individual and social factors, like values, emotions, and culture. The main predictors for behavioral intention differ from theory to theory and capture, for example, attitudes and expectations regarding the usefulness of the behavior and the effort to carry out the behavior. Mediators or moderators like age, gender, and the control one has over the behavior affect the strengths of the influences of the main predictors and behavioral intention on behavioral intention and behavior, respectively.

In this paper, we assess the prevalence and use of acceptance behavior theories and models in the software engineering literature by means of a mapping study [16,17]. This will provide a comprehensive overview of the evidence base of the research area and help other researchers in selecting and applying such theories for their contexts.

The remainder of the paper is structured as follows: Section 2, presents the related work for this study. Section 3 presents our research questions and research approach. In Section 4, we present the results and analysis of our study, which we further discuss in Section 5. In Section 6, we discuss threats to validity. Summary and conclusions (Section 7) conclude our paper.

2. Related work

Based on our earlier work [14], we identified two additional literature reviews [18,19] that are directly related to our study's purpose.

The review by Turner et al. [18] focused specifically on the use of one particular model of technology acceptance. Moreover, Turner et al. did not focus on the software engineering context.

Lorey et al. [19] reviewed the use of social science theories in software engineering more broadly without a focus on technology acceptance or acceptance behavior. They further analyzed in which knowledge areas the theories were used. However, their search was limited to five software engineering journals.

Börstler et al. [14] identified seven extensive literature reviews on technology acceptance and acceptance behavior from various fields. From these seven literature reviews, they identified 30 theories and models that are potentially relevant for research related to software development.

In this paper, we are looking specifically at theories and models of acceptance behavior (as identified by Börstler et al. [14], see Appendix A for a list) and how they have been used in research related to software development. We used an automated search to comprehensively cover at least the established list of 120 software engineering venues described in Section 3.1.2. In the identified studies, we analyze where the theories and models were used (i.e., in which knowledge areas) and how they were applied (i.e., which theoretical constructs of the theories and models were applied). This will help software engineering researchers in selecting a theory and appropriate constructs for their specific context. These differences are summarized in Table 1.

3. Method

To explore where, why, and how theories and models of acceptance behavior have been used in the software engineering literature, we

carry out a systematic mapping study [16,17]. The outcome of this study will provide a “map” of the area that shows publication trends, as well as commonalities of and differences between theories and studies. The map can, for example, be used to find a theory for investigating acceptance behavior in a particular knowledge area and how this theory has been operationalized. This will help software engineering researchers identify suitable theories for their specific context and apply them more effectively. The map can also be used to identify underexplored areas that indicate areas for further research.

We posed the following research questions:

- RQ 1. Which of the theories and models of acceptance behavior identified in [14] have been used in the software engineering literature?
- RQ 2. For which purposes have the theories been used?
- RQ 3. In which knowledge areas have the theories been used?
- RQ 4. How have the theories been operationalized?
 - RQ 4.1. Which data collection methods have been used?
 - RQ 4.2. Which constructs have been used?

3.1. Search strategy

We explored several search strategies, and after checking their feasibility (see Appendix C for a detailed description), we decided to use an existing list of relevant acceptance behavior theories (see Section 3.1.1) together with an established list of software engineering venues (see Section 3.1.2) and combined them into the following logical search string:

list of acceptance behavior theories AND

list of peer-reviewed software engineering publication venues

3.1.1. Acceptance behavior theories

For identifying keywords for theories of acceptance behavior, we have used the theories provided by Börstler et al. [14]. Their research, which was based on literature reviews in information systems, information technology, and the social and behavioral sciences, identified 30 models and theories of acceptance behavior that are potentially relevant for research related to software development. The complete list of venues can be found in Appendix A.

3.1.2. Software engineering venues

For the identification of a representative list of venues for software engineering research, we pursued the following approach:

- Karanatsiou et al.'s [20] list of journals and conferences related to software engineering. This list has been used in a series of articles where individual researchers and institutes have been ranked for their active contributions to software engineering research. We have used a more inclusive approach and included all venues categorized as “strictly relevant to the software engineering domain” by Karanatsiou et al. independently of their citation scores. We identified 76 software engineering journals and conferences from this list.
- Archambault et al.'s [21] list of software engineering journals. This classification of journals has been used in several science-wide evaluations of researchers' impact [22]. Archambault et al.'s list (version 1.06) comprises 59 journals categorized as software engineering.

Combining both lists yields 120 software engineering venues. A complete list of all venues can be found in Appendix B and an electronic supplement; see Section Data availability. In our search for the use of behavioral theories and models in software engineering, we relied on a search in a union of the venues in the two lists. For journals, where possible, the ISSN/e-ISSN number was used in the search. To improve the inclusiveness of the search and to ensure that all intended venues

Table 1

A comparison of the current study with related literature reviews.

	Turner et al. [18]	Lorey et al. [19]	Börstler et al. [14]	This paper
Aim	Evaluate if one particular model of technology acceptance predicts actual usage.	Prevalence of social science theories in software engineering.	Identify theories and models for technology acceptance and acceptance behavior.	Prevalence and use of the theories and models of acceptance behavior identified in [14] in software engineering.
Scope	Any field.	Software engineering.	Any field.	Software engineering.
Search coverage	Automated search in six digital libraries	Five software engineering journals.	Automated search in Scopus for surveys and reviews. Complemented with seminal reviews.	120 venues of software engineering research.
Aspects analyzed	Are the (chosen) acceptance model's constructs related to actual use?	Purpose of theory use and in which knowledge areas they are used.	Examples of identified theories and models from software engineering literature.	Purpose of theory use, in which knowledge areas they are used, and which theoretical constructs were applied in their use.

are found in the search, we did not use any keywords about the location (e.g., Australian, Asia-Pacific, European), sponsor/publisher (e.g., IEEE, ACM) or forums (e.g., conference, symposium) in the search. For example, instead of searching for “international conference on software engineering”, we have used the keyword “software engineering”. Thus, any venue (conference or journal) with software engineering in the title would be considered in our search. This means that our search becomes more inclusive than the combined lists of Karanatsiou et al. and Archambault et al.

3.2. Search engines and databases

As we were searching for papers in specific venues, it is sufficient to search in only one database if it indexes publications from these venues. We used Scopus because it has good coverage of the software engineering literature [23,24] and features to search for specific venues by venue names and ISSN identifiers. Furthermore, we consulted both the documentation of Scopus regarding their claimed coverage and manually confirmed for a subset of venues if the publications from those venues are indexed in Scopus.

Of the 120 venues in our list, only one was not indexed at all in Scopus as of March 14, 2023. A second one refers to an outdated name of an indexed venue, and a third one refers to a venue that we could not find even with Google or DBLP. We are therefore confident that using only Scopus provides sufficient coverage [16] for this mapping study.

3.3. Search string

We used the following search string on Scopus:

TITLE-ABS-KEY(<list of theories> **AND** <list of venues>)
AND (**LIMIT-TO**(LANGUAGE, “English”))
AND (**EXCLUDE**(DOCTYPE, “cr”))

Since our initial search result contained many conference reviews and summaries, we explicitly excluded those by adding ‘**EXCLUDE** (DOCTYPE, “cr”)’ to the search string. The full search string showing the operationalizations for all theories (see Section 3.1.1 for details) and all software engineering venues (see Section 3.1.2 for details) can be found in Appendix D.

3.4. Selection process

The selection process is summarized in Fig. 2, and the selection criteria are listed in Table 2. The search was carried out on May 09, 2023 and produced 197 unique papers. We screened these 197 papers by reading their titles, abstracts, and keywords using AbstrackR [25],

Table 2

Inclusion and exclusion criteria used for study selection.

Inclusion criteria	
IC1:	The paper is in the field of software engineering, <i>i.e.</i> , it is about the systematic application of scientific and technological knowledge, tools, methods, and experience to the design, implementation, testing, operation, maintenance, and documentation of software [26].
IC2:	The paper applies, evaluates, adapts, or proposes a theory for describing, explaining, or predicting the acceptance behavior of practitioners (<i>i.e.</i> , the entity of concern is involved in software development) regarding a software engineering phenomenon (<i>i.e.</i> , scientific or technological knowledge, tool, method, practice or experience).
Exclusion criteria	
EC1:	The paper is concerned with the acceptance behavior of software products by end-users or consumers.
EC2:	The paper is about process modeling, and it is obvious that the modeled process is not a software development process, phase, or practice.
EC3:	The paper is not written in English.

a tool that supports and automates the collaborative screening of titles, abstracts, and keywords.

Three of the co-authors performed a pilot round of selection based on titles, abstracts, and keywords. Ten studies were reviewed independently by each of the three authors. Four disagreements were resolved with discussion, and the selection criteria were further clarified. We decided that papers about process modeling will be excluded, where the process being modeled is not a software engineering process (e.g., business process modeling).

After the piloting, each paper was independently reviewed by at least two reviewers. The second author reviewed all 197 papers. This first selection round (based on titles, abstracts, and keywords) led to eight papers with conflicting ratings, *i.e.*, one author considered the paper relevant, and the other author considered it irrelevant to the current study. All eight papers were reviewed again, and a consensus was reached for each paper either to include it for full-text reading or to exclude it. For six of the eight papers, the agreement was that the paper must be read in full text. The remaining two papers were considered irrelevant to the current study. Based on titles, abstracts, and keywords, 74 papers were considered relevant for the study, and 123 papers were excluded.

We extracted the information for the fields mentioned in Table 3 from the 74 papers shortlisted after the first selection step. After reading the full text of these papers, we included 47 papers as primary studies and excluded another 27 papers. The full list of the 47 included

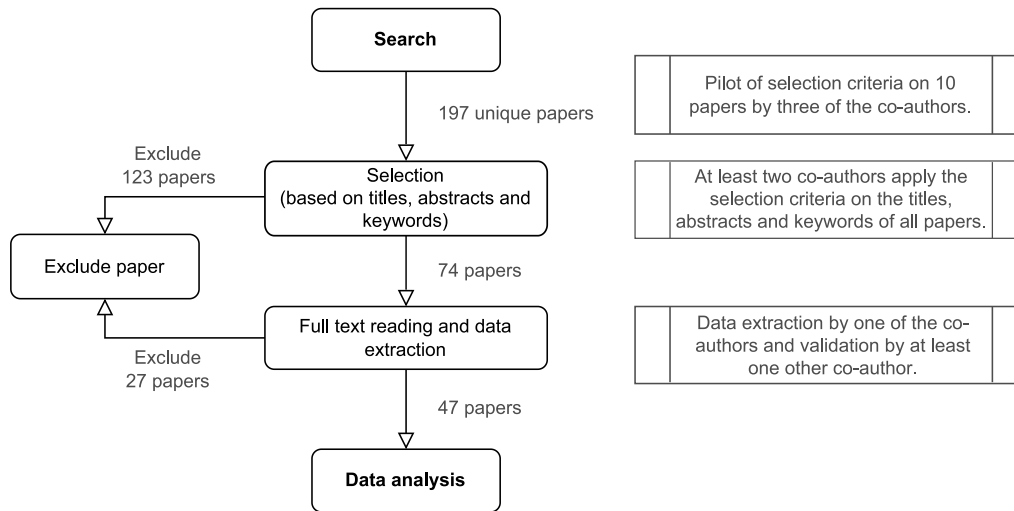


Fig. 2. An overview of the selection process and results.

Table 3
Description of data extraction fields.

Field	Description
ID	Identifier for the primary study.
Theory/ies	The theories and models used in the primary study. We used three columns to list the first, second, and further theories used in the primary study.
Purpose	Our interpretation of the primary study's description of the purpose of using said theory/ies (design, explain, or meta).
Topic/ context	Topic or context of the work, according to the 15 knowledge areas defined in SWEBOOKv3.
Means of collecting data	For example, interview or questionnaire.
Validation of data collection instrument	Kind of validation used for the data collection instrument, for example, Cronbach's alpha.
Aspects or constructs evaluated	Aspects or constructs that the primary study claims to evaluate, e.g., ease of use, usefulness, or attitude. The ordering is according to the importance given to a theory in the primary study. In cases of equal importance, we used the order of appearance.
Definition of evaluated aspects (if any)	As provided by the authors of the primary study.
Reflections on the theory (if any)	As provided by the authors of the primary study. For example, on applicability, the validity of constructs, or required/suggested changes.
Additional remarks	Comments by the researcher who extracted the data.

primary studies can be found in the end of the paper (after Section References). Information about the 27 papers excluded after full-text reading is available in an electronic supplement; see Section Data availability.

3.5. Data extraction and analysis

The data extracted from the full text of the studies is described in Table 3. For full-text reading and data extraction, the 74 papers shortlisted after the first selection step were distributed among the four co-authors. Papers that were excluded by one of the co-authors in this step were discussed among the first and the second authors until an agreement was reached. This decision was then forwarded to the other co-authors and discussed further when necessary. These steps led to the exclusion of 27 papers.

For the remaining 47 papers, each co-author validated two fields. Deviations were discussed with the co-author, who extracted the data until an agreement was reached. After the validation, the data was also cleaned and coded by the validating author (e.g., regarding the SWEBOOK knowledge area and purpose of theory usage). Another co-author then reviewed the coding. The filled-in data extraction form for the 47 included primary studies is available in an electronic supplement; see Data Availability section.

In the included studies, the purposes of using the theories are aligned with the overarching aims of the studies. Thus, to answer RQ2, we coded the purposes based on the positioning made by the authors of the studies. Furthermore, we coded the purposes as contribution types in accordance with the design science paradigm [27] and identified three broad categories of purposes:

1. **Design.** This category comprises studies using theories to guide the design of a new technology or way of working.
2. **Explain.** This category comprises explanatory research. It includes studies using theories to assess or predict the likelihood of adoption or acceptance of a new technology or way of working in context.
3. **Meta.** This category comprises studies that evaluate or compare the theories as such, develop new theories (e.g., studies that propose new theories or identify new constructs, propositions, explanations, or scope for a theory), or contribute to theory implementation (e.g., studies in which the development of new instruments for data collection to operationalize a theory is the contribution).

To answer RQ4.2, we extracted the constructs or aspects as described in the studies. We then applied data cleaning to remove spelling variations and synonyms. After cleaning the data, one author iteratively

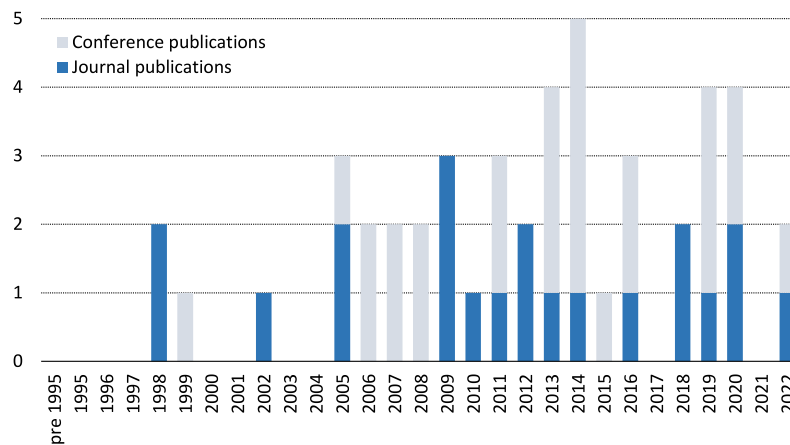


Fig. 3. Included primary studies by type of publication venue (conference/journal) and publication year (n=47).

“unified” the list of constructs by grouping semantically similar constructs. The results were then reviewed together with another author. For example, perceived usefulness, as defined in TAM, was considered semantically similar to relative advantage, as defined in Perceived Characteristics of Innovating (PCI). If available, we considered the studies’ definitions of constructs in this grouping. Since it was sometimes difficult to assess whether a study evaluated a construct or the perception of it, we decided to group the following constructs together: perceived usefulness and usefulness, and perceived ease of use and ease of use.

4. Results and analysis

4.1. Distribution of primary studies over time

Our search and selection process found 47 relevant primary studies published between 1998 and 2022 (see Fig. 3 for an overview). We did not find any primary studies published before 1998, although most theories we searched for were proposed in 1992 or earlier. Of the theories that have been used more than once, only Task-Technology-Fit (TTF [28]) has been proposed after 1992, i.e., in 1995. Similarly, among the theories that have been used in only a single primary study, two theories were proposed after 1992 (UTAUT, 1998; and a hybrid model presented in one of the primary studies, [P3], in 2006). There is a fairly even distribution between journal publications (21) and conference publications (26). Since 2005, the number of publications per year has mostly been in the range of 2–3. Despite recent interest in theoretical foundations [14,19], there has not been a recent increase in theory use.

4.2. Identified acceptance behavior theories (RQ1)

We identified five groups of acceptance behavior theories that have been used in more than one primary study in the software engineering literature: (1) *Technology acceptance model (TAM)*, (2) *Innovation theories*, (3) *Theory of planned behavior/reasoned action (TPB/TRA)*, (4) *Fitness models*, and (5) *Social cognitive theory*. These five theory groups are briefly described in the following subsections (4.2.1–4.2.5).

Seven more theories have been used in a single primary study, each, and have been grouped together in theory group *Other*: A custom-designed hybrid model, the IS success model, a motivation model, the Technology adoption framework, the Model of PC utilization (MPCU), the Unified Theory of Acceptance and Use of Technology (UTAUT), and Social Capital Theory. Four of the primary studies in this group used theories of the five other groups as well.

In total, we identified 56 usages of a theory group in the 47 included primary studies (see Table 4). Five of these studies ([P3], [P5], [P6], [P15], [P17]) used theories of 2–4 theory groups each. [P17], for

example, even used five theories: TAM and TAM2 (which we counted as one occurrence of theory group TAM), Perceived Characteristics of Innovating (PCI, in group Innovation), TPB, and MPCU (in group Other). All studies using multiple theories included TAM.

4.2.1. Technology acceptance model

The Technology Acceptance Model (TAM) was proposed by Davis [29,30] as an extension of the Theory of Reasoned Action (TRA, see Section 4.2.3 and Fig. 5) [31]. Whereas TRA is context-agnostic and applicable to human behavior in general, TAM has been specifically adapted to the context of technology adoption. In TAM, TRA’s general construct “attitude toward behavior” has, for example, been replaced with the more technology-oriented constructs “perceived usefulness” and “perceived ease of use”. Since its development, TAM has been successively extended, refined, and adapted (e.g., TAM2 [32], TAM3 [33] and the Method Evaluation Model (MEM) [34]) and has been applied widely in technology adoption research [35].

Of the 47 included primary studies, 29 (61.7%) used TAM or one of its extensions. Since it was not always clear which version of TAM was used, we did not distinguish between different versions of TAM. One study used two versions of TAM ([P17]), which we counted as one occurrence of TAM, though.

4.2.2. Innovation theories

Innovation theories propose a staged adoption model to explain the implementation, adoption, and dissemination of innovations. The five stages in Rogers’ Diffusion of Innovations theory [36] lead from initially becoming aware of an innovation to the confirmation of an adoption decision. An overview of the five stages is shown by the arrows in Fig. 4. An adoption decision is made in stage “Adoption decision” (middle arrow) and is influenced by five main “perceived characteristics of the innovation”: Relative advantage, compatibility, complexity, trialability, and observability.

Of the 47 included primary studies, 10 (21.3%) used an innovation theory.

4.2.3. Theory of planned behavior/reasoned action

The Theory of Planned Behavior (TPB) [37] is an extension of the Theory of Reasoned Action (TRA) [31]. Both theories postulate that an individual’s beliefs about behaviors and norms influence the individual’s subjective norms and behavioral attitudes. The latter two influence behavioral intention, which is used as a predictor for behavior. TPB adds perceived behavioral control since individuals may lack control over the targeted behavior (see Fig. 5). TPB and TRA are widely used in the health sciences, social sciences, and psychology.

Of the 47 included primary studies, 6 (12.8%) used TPB/TRA.

Table 4

Theories used in the 47 primary studies. The total count is larger than the total number of studies since several studies employ or combine several theories and models.

Theory group	Count	Studies
TAM	29	[P1], [P3], [P5], [P6], [P8]–[P11], [P14], [P15], [P17]–[P19], [P22], [P27]–[P31], [P33], [P36], [P39], [P40], [P42]–[P47]
Innovation	10	[P6], [P12], [P13], [P16], [P17], [P24]–[P26], [P34], [P35]
TPB/TRA	6	[P3], [P6], [P17], [P37], [P38], [P41]
Fitness	2	[P2], [P32]
Social Cognitive Theory	2	[P4], [P23]
Other	7	[P3], [P5], [P7], [P15], [P17], [P20], [P21]
TOTAL	56	

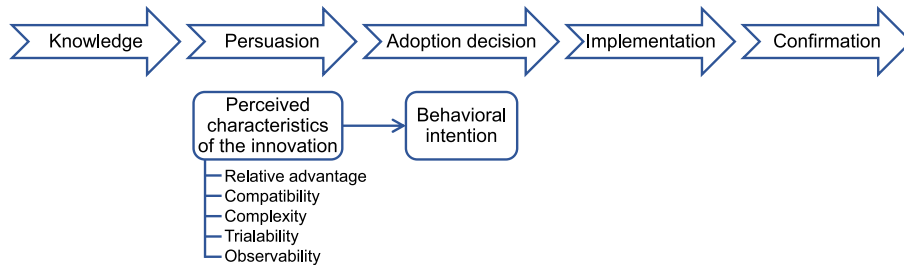


Fig. 4. The five stages of Roger's diffusion of innovation theory (arrows) and relevant constructs (rounded boxes).

Source: Adapted from [36].

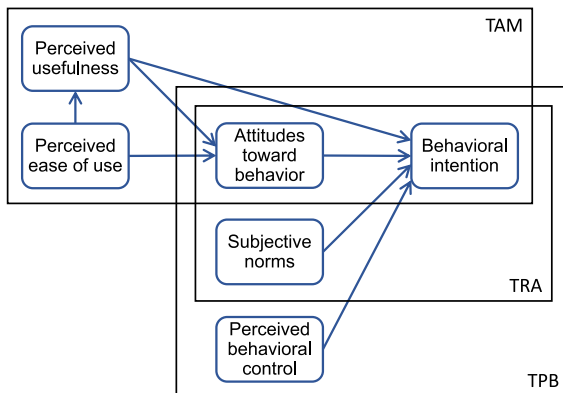


Fig. 5. Key constructs, and their relationships¹, of TAM, TRA, and TPB. The figure for TAM is based on Davis' early work [29,30]. Attitude was dropped in later publications/versions of TAM.

4.2.4. Fitness models

Fitness models provide a task-oriented view of technology acceptance. They focus on a technology's "fitness" for supporting a user's needs to perform a particular task. Since specific tasks may have specific characteristics, fitness models need to be adapted to the context in which they are used. This task-oriented view complements other views of technology acceptance. It is therefore suggested to integrate fitness models with common technology acceptance models, like TAM [38].

One primary study (S12) used two fitness models (Task-technology-fit and Fitness-for-use) that were later integrated into a hybrid fitness model. Since all fitness models were categorized into one group ("Fitness"), we counted S12's use of fitness models only once.

Of the 47 included primary studies, 2 (4.3%) used a fitness model.

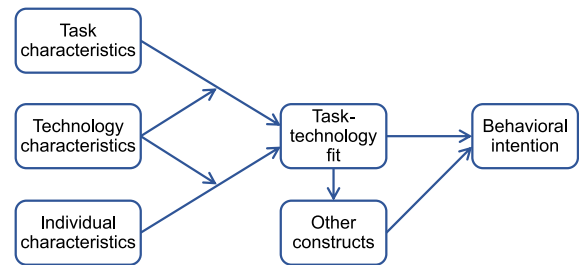


Fig. 6. Key constructs of fitness models.

Source: Adapted from [28].

4.2.5. Social cognitive theory

Social cognitive theory is an extension of social learning and is based on the concept of observing behaviors and their consequences in a social context [39]. A person's past experience, his/her self-efficacy, and the behavior's expected outcomes (personal as well as social) determine the likelihood of this person's engagement in the behavior.

A unique and influential construct of Social cognitive theory is self-efficacy which describes the level of a person's confidence in his/her ability to successfully perform a behavior. Self-efficacy has been widely adopted in other theories of behavior.

Of the 47 included primary studies, 2 (4.3%) used Social cognitive theory. However, none of them operationalized its use by means of specific constructs.

4.3. Purpose of using the theories (RQ2)

We identified two overarching purposes of using the acceptance behavior theories (see Table 5) in software engineering: (1) guiding the *design* of software interventions/artifacts and (2) to *explain* (or predict) acceptance (or lack of acceptance) of software interventions/artifacts. A few primary studies took a *meta* perspective and studied the theories as such. Of our 47 primary studies, 20 primary studies used acceptance

¹ In Figs. 5–7 an arrow from construct A to construct B indicates that construct A influences construct B. An arrow from a construct to an arrow indicates that this construct mediates the influence of the arrow it points at.

Table 5

Purpose of use for technology acceptance models or theories in the 47 primary studies. Rows listing several theory groups refer to primary studies that use multiple theories.

Theory group	Purpose	Count	Studies
TAM	Design	5	[P14], [P19], [P42], [P43], [P45]
	Explain	17	[P1], [P9]–[P11], [P18], [P22], [P28]–[P31], [P33], [P36], [P39], [P40], [P44], [P46], [P47]
	Meta	2	[P8], [P27]
TAM, Other	Design	1	[P5]
TAM, Innovation, TPB/TRA	Explain	1	[P6]
TAM, Other	Meta	1	[P15]
TAM, TPB/TRA, Other	Meta	1	[P3]
TAM, Innovation, TPB/TRA, Other	Meta	1	[P17]
Innovation	Design	5	[P12], [P24], [P25], [P34], [P35]
	Explain	1	[P26]
	Meta	2	[P13], [P16]
TPB/TRA	Design	2	[P38], [P41]
	Meta	1	[P37]
Fitness	Design	1	[P32]
	Explain	1	[P2]
Social Cognitive Theory	Design	1	[P23]
	Meta	1	[P4]
Other	Design	2	[P7], [P20]
	Meta	1	[P21]

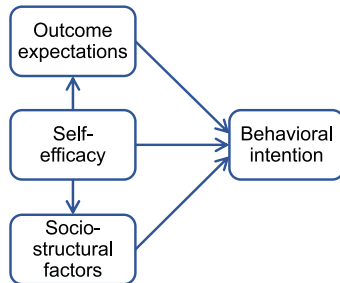


Fig. 7. Key constructs of Social Cognitive Theory.
Source: Adapted and simplified based on [40].

behavior theories for explanatory purposes, 17 used acceptance behavior theories for design purposes, and the remaining 10 took a meta perspective.

Most explanatory studies (18) in our sample use TAM and only two explanatory studies do not use TAM. However, many of them do not investigate acceptance behavior but do mainly assess the usefulness and ease of use of a “technology”. One could therefore argue that they are mere usability assessments rather than studies of acceptance behavior. Whereas TAM was most often used for explanatory purposes (18 out of 29 studies), theories of innovation were most often used for design purposes (5 out of 8 studies).

In many cases, theories are reflected on at a general level but they are not instantiated and applied, e.g., Social Cognitive Theory. Only 46 of 56 “usages” of a theory explicitly apply any of its constructs (see Table 11).

4.4. Knowledge areas of using the theories (RQ3)

Table 6, shows which knowledge areas have been investigated in the 47 primary studies and Fig. 8 provides a graphical mapping of knowledge areas along the theory groups used to study these knowledge areas. Software construction has been studied most often (12 times) and four theory groups have been used to study this area. Software engineering process and software engineering models and methods were the next most often studied knowledge area (nine times, each). Although, software engineering professional practice was studied only six times,

it was studied using five of the six theory groups, whereas software engineering models and methods were studied only using TAM and innovation theories. Software configuration management and SWEBOK’s three foundation areas (computing, mathematical-, and engineering foundations) were not studied in any of the primary studies.

4.5. Operationalization of the theories (RQ4)

4.5.1. Data collection methods (RQ4.1)

Table 7 provides a detailed overview of which data collection methods have been used together with which theories. As the summary in Table 8 shows, questionnaires are used in 33 of the 42 primary studies that collect some form of specific data, followed by interviews, which are used in 9 primary studies. Five studies did not collect any specific data. Table 7 also shows that many primary studies used more than one data collection method. Although questionnaires are by far the most common data collection method, questionnaires have not been used in studies that just use TPB/TRA or Social Cognitive Theory.

Of the 42 primary studies that collect some form of specific data, 22 used a reference to another publication as a “validation” of their data collection instrument (see Table 9). Eight studies conducted an explicit validation. The remaining 17 studies either did no validation (12 studies), or did not need a validation, since they did not collect specific data (5 studies).

4.5.2. Constructs used (RQ4.2)

Initially, three of the authors extracted 77 unique constructs formulated in the primary studies. The data cleaning reduced the number of constructs to 57. Of those 57 constructs, 37 were used only once, i.e., in a single study. Of the remaining 20 constructs, 12 were used twice and 8 more than two times. The iterative grouping resulted in the 18 groups shown in Table 10.

Table 11 summarizes the constructs used in the primary studies (grouped by construct groups, see Table 10) and relates them to the theories used in the primary studies. The summary shows that different groups of theories focus on different aspects of acceptance behavior. Four of the 18 constructs (*perceived ease of use*, *intention to use*, *compatibility*, and *perceived behavioral control*, marked in dark yellow) are used in all groups of theories. This indicates that there are “universal” constructs that are important for investigating acceptance behavior in software engineering, irrespective of the underlying theory. Another

Table 6

Knowledge areas studied using technology acceptance models or theories in the 47 primary studies. Rows listing several theory groups refer to primary studies that use multiple theories. In total, there are 56 theory usages in the 47 primary studies.

Theory group	SWEBOK knowledge area	Count	Studies
TAM	Software Requirements	3	[P11], [P29], [P36]
	Software Design	1	[P22]
	Software Construction	3	[P14], [P28], [P39]
	Software Testing	1	[P42]
	Software Maintenance	1	[P10]
	Software Engineering Management	3	[P1], [P19], [P27]
	Software Engineering Process	1	[P33]
	Software Engineering Models and Methods	7	[P8], [P9], [P18], [P43]–[P47]
	Software Quality	2	[P30], [P40]
	Software Engineering Professional Practice	1	[P31]
	Software Engineering Economics	1	[P47]
TAM, Other	Software Construction	1	[P5]
	Software Quality	1	[P15]
TAM, TPB/TRA, Other	Software Construction	1	[P3]
TAM, Innovation, TPB/TRA	Software Engineering Process	1	[P6]
TAM, Innovation, TPB/TRA, Other	Software Engineering Process	1	[P17]
Innovation	Software Requirements	1	[P26]
	Software Design	2	[P13], [P24]
	Software Construction	2	[P12], [P34]
	Software Engineering Models and Methods	2	[P16], [P35]
	Software Engineering Professional Practice	1	[P25]
TPB/TRA	Software Engineering Process	1	[P38]
	Software Engineering Professional Practice	2	[P37], [P41]
Fitness	Software Maintenance	2	[P2], [P32]
Social Cognitive Theory	Software Design	1	[P23]
	Software Engineering Professional Practice	1	[P4]
Other	Software Construction	2	[P7], [P20]
	Software Engineering Professional Practice	1	[P21]

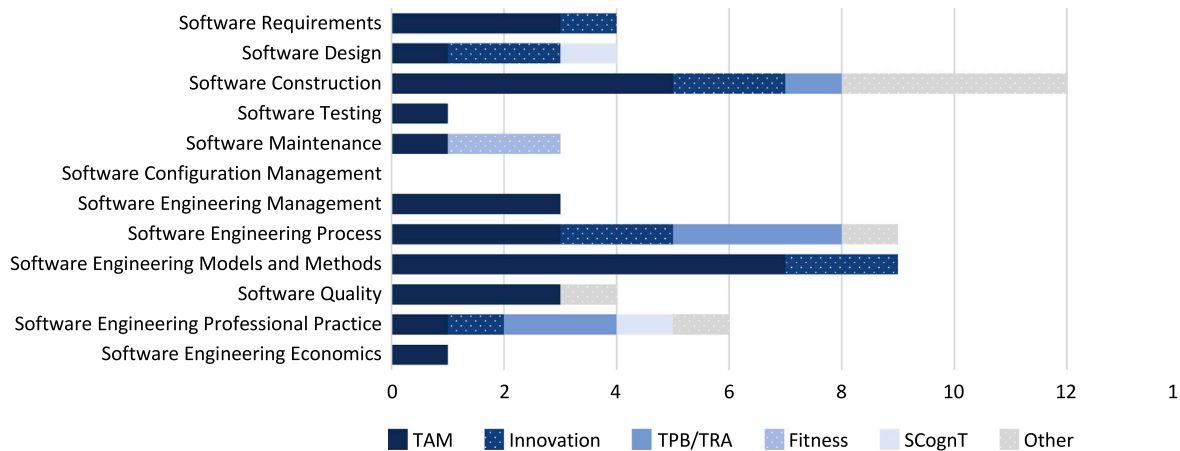


Fig. 8. SWEBOK's 12 knowledge areas dealing with software engineering by the theory groups used to study them. SWEBOK's 3 foundation areas (Computing-, Mathematical-, and Engineering Foundations) have not been studied and are omitted in the figure. In total, there are 56 theory usages in the 47 primary studies.

four of the 18 constructs (*attention*, *satisfaction*, *price value*, and *organizational factors*, marked in light yellow) are used only in a single group of theories. This indicates that there might be no one-size-fits-all theory for investigating acceptance behavior in software engineering.

It can also be noted that only small subsets of constructs are commonly used by all or almost all primary studies within any of the groups (cells in dark green). In the case of TAM, for example, it can be noted that all studies use *perceived usefulness* and *perceived ease of use*. However, only about half (14 of 27) use *intention to use*, although all three constructs have been present as core constructs in TAM since its inception [30].

Another observation from our data is that, in many cases, theories are reflected on at a general level, but the theories are not instantiated and applied, for example, for the two occurrences of Social Cognitive Theory (group SCogNT). In those cases, the theories do not use any explicit constructs. In total, 10 of the 56 theory “usages” do not use any explicit constructs. This issue is most common in the Innovation group, where 6 of the 10 uses lack explicit constructs.

5. Discussion

Using a systematic mapping study, this study explored the prevalence of major theories and models of acceptance behavior in the software engineering literature and how they have been used.

Our data indicates that TAM is the most influential theory of acceptance behavior in the software engineering context. Our data, furthermore, shows that different theories focus on different sets of constructs (cells marked green in Table 11). However, on the other hand, we can see that there is a group of four constructs that is used in all theory groups (marked dark yellow in Table 11). This means that there are commonalities between the theories but that different theories focus on slightly different aspects of acceptance behavior and that there might be no single (one-size-fits-all) theory that can be used to investigate all aspects of acceptance behavior that are relevant in software engineering contexts.

The variation of used constructs is large, even within theory groups. This indicates that the studies in our sample take some liberties in their

Table 7
Data collection methods by theory group and study.

Theories	Data collection method	Count	Studies
TAM	Questionnaire	21	[P1], [P9]–[P11], [P18], [P19], [P22], [P27]–[P31], [P33], [P36], [P39], [P40], [P42]–[P45], [P47]
	Questionnaire & interviews	1	[P8]
	Questionnaire & focus groups	1	[P46]
	NA*	1	[P14]
TAM, Other	Questionnaire	1	[P5]
TAM, Innovation, TPB/TRA, Other	Questionnaire	1	[P17]
TAM, TPB/TRA, Other	Questionnaire & interviews	1	[P3]
TAM, Other	NA	1	[P15]
TAM, Innovation, TPB/TRA	NA	1	[P6]
Innovation	Focus groups	1	[P16]
	Interviews	2	[P34], [P35]
	Questionnaire	3	[P12], [P13], [P24]
	Repository-/archival analysis	1	[P25]
	NA	1	[P26]
TPB/TRA	Interviews	2	[P37], [P38]
	Interviews & focus groups	1	[P41]
Fitness	Questionnaire	2	[P2], [P32]
Social Cognitive Theory	Interviews	1	[P23]
	NA	1	[P4]
Other	Interviews	1	[P20]
	Questionnaire	2	[P7], [P21]

*NA: Not applicable, no specific data was collected.

Table 8
Data collection methods used in the 42 primary studies (out of 47) that did collect specific data. The total count is larger than 42 since several studies use multiple methods.

Data collection method	Count	in % ^a
Questionnaire	33	78.6%
Interview	9	21.4%
Focus group	3	7.1%
Repository-/archival analysis	1	2.4%
TOTAL	46	

^a Of the studies that collected data (42 of 47).

Table 9
Validation approaches used for the data collection instruments.

Validation approach	Count	Studies
Instrument is based on a reference	22	[P2], [P3], [P7], [P8], [P10], [P17], [P18], [P21], [P22], [P28], [P29]–[P33], [P36], [P37], [P40]–[P43], [P47]
No validation	12	[P1], [P11]–[P13], [P16], [P19], [P20], [P23], [P25], [P34], [P35], [P38]
Cronbach's alpha	4	[P9], [P27], [P39], [P45]
Cronbach's alpha & factor analysis	2	[P44], [P46]
Internal reviews	1	[P24]
Piloting with students	1	[P5]
NA*	5	[P4], [P6], [P14], [P15], [P26]

*NA: Not applicable, no specific data was collected.

usage of the theories. As described in Section 3.5, our mapping identified 77 constructs, of which many are synonyms or closely related (see Table 10 for a summary). Even after aligning construct descriptions, there is a wide variety of construct types used when applying the theories. Furthermore, many studies lack clear definitions of the constructs they use or they use sets of constructs that are not fully aligned with the theories they claim to be using. This leads to a proliferation of constructs and makes it difficult to draw valid conclusions from theory usage. It also makes it more difficult to compare the results between studies, even when the studies are conducted within the same area using the same underlying theory. The proliferation of constructs has also been discussed in Börstler et al. [14, Sect. 4].

A lack of validated data collection instruments exacerbates this problem. Such a validation investigates whether a data collection instrument measures what it is supposed to measure. Since the contextual

factors in software development contexts might differ from the theories' original contexts, validation within software development contexts might be needed. Which form of validation is applied is less of a concern. However, relying on existing instruments that might not have been validated in software development contexts might be an issue.

In our sample, software construction and software engineering models and methods were the most frequently studied knowledge areas, with nine primary studies each, including five times using TAM ([P3], [P5], [P14], [P28], [P39]) and seven times using TAM [P8], [P9], [P18], [P43]–[P47]), respectively. An overview of the constructs used in those primary studies can be found in Table 12 and Table 13, respectively.

In the sample studies using TAM, it can be noted that all studies measure *perceived usefulness* and *perceived ease of use*. However, only about half of them (14 of 27) measure *intention to use*, although all

Table 10

Grouping of constructs after data cleaning. The groups and the constructs within groups are in alphabetical order.

Group representative	Constructs
Acceptance	Acceptance, Actual use, Current usage
Attention	Attention
Attitude	Attitude
Career consequences	Career consequences
Compatibility	Compatibility, Completeness, Correctness, Coverage, Perceived compatibility, Perceived quality, Production fit, Task-tool fit, Technological factors, Understandability
Competence	Competence, Confidence, Experience, Technology flexibility
Complexity	Complexity, Effort expectancy
Coordination fit	Coordination fit, Horizontal relationships, Market relationships, Vertical relationships
Effectiveness	Actual efficacy, Effectiveness, Improvement potential, Net benefits, Perceived productivity, Performance expectancy, Project performance, Task performance
Intention to use	Intention to use
Organizational factors	Organizational factors
Perceived behavioral control	Behavioral control, Business flexibility, Facilitating conditions, Perceived behavioral control, Perceived maturity of tools, Voluntariness
Perceived ease of use	Ease of use, Perceived ease of use
Perceived usefulness	Usefulness, Perceived usefulness, Relative advantage, Relevance
Price value	Price value (cost)
Results demonstrability	Observability, Results demonstrability, Testability, Visibility
Satisfaction	Satisfaction
Subjective norm	Image, Social influence, Subjective norm

Table 11

Constructs used in the 47 primary studies grouped by construct groups (see Table 10) and theory groups (see Table 4). The constructs are sorted in descending order by the frequency of their application, see column *TOT* to the right. SCogNT = Social Cognitive Theory.

	Theory groups						
	TAM	Innovation	TPB/TRA	Fitness	SCogNT	Other	TOT
Theory usages	29	10	6	2	2	7	56
Usages that apply constructs	27	6	5	2	0	6	46
Applied constructs per theory group							
Perceived usefulness	27	5	2	0	0	4	38
Perceived ease of use	27	2	2	1	0	3	35
Intention to use	14	1	2	1	0	2	17
Compatibility	5	5	1	2	0	1	14
Perceived behavioral control	3	1	3	1	0	4	12
Subjective norm	3	1	5	0	0	3	12
Effectiveness	6	1	0	0	0	3	10
Attitude	2	1	4	0	0	1	8
Results demonstrability	2	3	1	0	0	1	7
Acceptance	2	1	0	1	0	1	5
Career consequences	1	1	1	0	0	1	4
Competence	1	1	0	0	0	2	4
Complexity	0	2	0	0	0	1	3
Coordination fit	0	0	0	1	0	1	2
Attention	0	0	0	0	0	1	1
Satisfaction	0	0	0	0	0	1	1
Price value (cost)	0	0	0	0	0	1	1
Organizational factors	0	1	0	0	0	0	1
No. of applied constructs	12	14	9	6	0	17	18
Legend:							
Used in all groups that apply constructs.	Used in all studies in this group.						
Used in one group only.	Used in at least 50% of the studies in this group.						

three constructs have been included in TAM since its inception [30]. Seven of the 21 studies used only *perceived usefulness* and *perceived ease of use*. Without also measuring *intention to use*, the relationships of other measured constructs to intention to use (behavioral intention) or actual behavior (acceptance) cannot be evaluated. If conclusions are drawn about acceptance behavior without controlling for intention to use, the validity of these conclusions is questionable. Beyond the three central TAM constructs (*perceived usefulness*, *perceived ease of use*, *intention to use*), nine further constructs are used by 1–6 studies (of 27). Of those nine, one (*effectiveness*) is not used in any theory in the TAM group, i.e., the versions of TAM (TAM, TAM2, and TAM3) or the Method Evaluation Model.

From the descriptions of the theories in Section 4.2, we can see that behavioral intention (intention to use) is a key construct in all theories of acceptance behavior. Behavioral intention is commonly used as a predictor or proxy for actual behavior [15], although this relationship is considered weak [41,42]. It is, therefore, surprising that only 20 of the 56 (35.7%) theory uses in our sample studies use this construct. This might indicate that most studies in our sample do not investigate acceptance behavior and use the theories for other purposes.

We now briefly revisit the three literature reviews [14,18,19], presented in Section 2.

Regarding TAM, our findings that a majority of studies in the software engineering literature do not assess the construct of behavioral

Table 12

Constructs measured and validation approaches used in the five TAM-studies in knowledge area software construction.

	[P3]	[P5]	[P14]	[P28]	[P39]
Publication year	2006	2010	2009	2016	2014
Validation approach ^a	Reference	Pilot	No validation	Reference	CA
Perceived usefulness	X	X	X	X	X
Perceived ease of use	X	X	X	X	X
Intention to use	X	X		X	X
Compatibility					X
Perceived behavioral control	X				
Subjective norm	X				
Effectiveness		X			
Attitude	X				X
Acceptance		X			
Satisfaction		X			

^a CA = Cronbach's alpha.**Table 13**

Constructs measured and validation approaches used in the seven TAM-studies in knowledge area software engineering models and methods.

	[P8]	[P9]	[P18]	[P43]	[P44]	[P45]	[P46]
Publication year	2013	2005	2016	2020	2016	2022	2019
Validation approach ^a	Reference	CA	Reference	Reference	CA&FA	CA	CA&FA
Perceived usefulness	X	X	X	X	X	X	X
Perceived ease of use	X	X	X	X	X	X	X
Intention to use	X	X					X
Compatibility	X	X					
Subjective norm	X						
Effectiveness			X				
Results demonstrability		X					
Acceptance	X						

^a CA = Cronbach's alpha, FA = Factor analysis.

intention and only assess perceived ease of use and perceived usefulness is alarming when seen with the findings of Turner et al. [18]. As Turner et al. found in their analyzed studies, behavioral intention was a better predictor of acceptance.

Because of more comprehensive coverage of the software engineering literature than Lorey et al. [19], whose search was limited to five software engineering journals, we found more evidence of the use of theories of acceptance behavior in software engineering. However, the order of the most used theories in the literature is the same in both studies. The results differ in the distribution of identified papers to investigated knowledge areas.

Lastly, we have capitalized Börstler et al. [14] results and shown how major theories and models for acceptance behavior have been used in research related to software development.

6. Limitations

In this section, we discuss validity threats and limitations following Ampatzoglou et al.'s [43] classification of threats to validity in secondary studies in software engineering.

6.1. Study selection validity

Validity threats in this category are related to the search and selection processes for identifying relevant primary studies. In this study, we draw conclusions on the usage of acceptance behavior theories in the software engineering literature. Since the topic is interdisciplinary, there is a risk of missing relevant literature published in non-software engineering venues. Similarly, within the software engineering venues, there is a risk of missing relevant literature due to non-conventional usage of the theories and lack of alignment in terminology. Furthermore, as we only included publications written in English, there is a risk of missing relevant literature in other languages. We did not put any time restrictions on the search but may have missed relevant literature

indexed after May 9, 2023. The validity threat of missing relevant non-peer-reviewed literature was not considered a risk in this study since the use of acceptance behavior theories has not yet reached common software engineering practice and could thus still be considered mainly an academic exercise.

To mitigate the risk of exclusion bias, we ensured to use our selection criteria consistently. This was done by piloting and improving the selection criteria on a common set of 10 studies and, for the rest of the studies having at least two reviewers for each study.

6.2. Data validity

Validity threats in this category are related to data extraction and analysis. Or research questions regarded which theories have been used (RQ1), the purposes of using them (RQ2), the knowledge area in which they have been used (RQ3), and how they were operationalized (RQ4). While “what” questions, i.e., RQ1 and RQ3, were easily extracted from the studies, using explicit mentions of theories and a well-defined set of software engineering knowledge areas, the questions of “how” (RQ4) and “why” (RQ2) required more interpretation from the reviewers and thus entail a higher risk of interpretation bias or inconsistency in the data extraction.

For secondary studies in general, there is also a validity threat related to the quality of the primary studies. However, since we are not drawing any conclusions regarding the effect or value of using the theories, we did not consider this a threat in our study. For the same reason, we do not consider publication bias a threat.

6.3. Research validity

Validity threats in this category are related to overarching threats concerning the overall study design. The interdisciplinary nature of the topic introduces a risk of researchers not being familiar enough with the topic to draw incorrect conclusions about the theory usage.

In this study, all researchers are from the software engineering domain. However, the identification of relevant theories was made in a previous study as a collaboration between some of the co-authors and a researcher with a background in economics and behavioral sciences [14].

7. Summary and conclusions

We conducted a mapping study to examine the use of acceptance behavior theories in the software engineering literature. Our analysis of 56 theory uses in 47 primary studies identified five main groups of theories that have been used to investigate acceptance behavior in software engineering. Technology acceptance models (TAM and its extensions) were used in 29 (61.7%) of the 47 primary studies, innovation theories in ten (21.3%), and the theories of planned behavior/ reasoned action (TPB/TRA) in six (12.8%). All other theories (including Fitness and Social Cognitive Theory) were used in at most two of the primary studies.

There is little overlap between the key constructs between the theory groups, suggesting that they serve different purposes and are applied in different situations. Our mapping indicates that in the software engineering literature, theories, and constructs are often applied liberally and are highly customized to specific application contexts. This makes it difficult to compare results, even when studies use the same theories in similar contexts.

Despite the central role of the construct *Intention to use* (behavioral intention) in most acceptance behavior theories as a predictor for usage (or behavior), we found that it was used in only 17 (37%) of the 46 theory usages in our sample that applied specific constructs. This raises the question of whether the remaining studies were truly investigating acceptance behavior or using the theories for other purposes.

Furthermore, few studies explicitly validated their data collection instruments. Therefore, the current operationalization of acceptance behavior theories in empirical studies in software engineering seems to undermine the purpose of using these theories, which is to provide a transferable and comparable framework for understanding acceptance behavior. Consistency in the use of instruments for acceptance behavior theories, such as TAM is critical for producing reliable and comparable results. As Turner et al. [18] pointed out, “[c] are should be taken when using the TAM outside the context in which it has been validated” to ensure that the results are still meaningful. Therefore, a similar analysis to that of Turner et al. could be conducted for other acceptance theories to determine their generalizability across different contexts.

We believe that academics have a crucial role to play in making acceptance behavior theories more accessible, customized, and validated so that software engineering practitioners can more confidently adopt research results based on these theories.

We propose the following strategies to mitigate the issues discussed in this paper:

- “Stick to the theory” and the definition of constructs it provides.
- Measure all constructs, in particular *intention to use* (behavioral intention).
- Use available data collection instruments.
- Validate the data collection instruments when they are not validated in the current context.
- Check the strengths of relationships between the theory’s core constructs and *intention to use*.
- If possible, measure actual use (behavior). This will make it possible to evaluate how well a theory predicts actual use.
- Provide sufficient contextual information. This will make other researchers aware of the external factors and mediators that might have influenced the core constructs and relationships (see Fig. 1).

Taken together, these strategies will help to increase the validity and reliability of software engineering research. It will also make it easier to aggregate study results and facilitate meta-analyses.

CRediT authorship contribution statement

Jürgen Börstler: Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Nauman bin Ali:** Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Kai Petersen:** Writing – review & editing, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Emelie Engström:** Writing – review & editing, Validation, Methodology, Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The following data pertaining to this study is available on Zenodo (<https://doi.org/10.5281/zenodo.8060722>): (1) the list of software engineering venues, (2) the search string, (3) the list of the 47 included studies, (4) the data extracted for the 47 included studies, and (5) the list of the 27 studies excluded during full-text reading and data extraction (together with rationales for exclusion).

Acknowledgments

This work has been supported by ELLIIT; the Strategic Research Area within IT and Mobile Communications, Sweden, funded by the Swedish Government. The work has also been supported by a research grant for the GIST project, Sweden (reference number 20220235) from the Knowledge Foundation in Sweden.

Appendix A. List of theories (based on [14])

Big Five theory
 Compatibility-UTAUT (C-UTAUT)
 Decomposed Theory of Planned Behavior
 Diffusion of Innovations Theory
 Expectation Confirmation Theory
 Igarbaria’s model
 Information-Motivation-Behavioural skills model
 Information Systems success model/ IS success model
 Innovation Diffusion Theory
 Inter-Organizational Relationship theory
 Model of Acceptance with Peer Support
 Model of PC Utilization (MPCU)
 Motivational model
 Perceived Characteristics of Innovating (PCI)
 Perceived value model
 Social capital theory
 Social Cognitive Theory
 Social identity theory
 Stages of Change Model
 TAM Extension (TAME)
 Task-Technology Fit model (TTF)
 Technology Acceptance Model (TAM)
 Technology-organization-environment framework
 Technology Integration Model
 Theory of Interpersonal Behaviour

Theory of planned behavior (TPB)
 Theory of reasoned action (TRA)
 Unified Theory of Acceptance and Use of Technology (UTAUT)
 Uses and Gratification theory (U&G)

Appendix B. List of software engineering venues (based on [20, 21])

ACM Conference On Object Oriented Programming Systems Languages And Applications
 ACM Siggraph Computer Graphics
 ACM Sigplan Notices
 ACM Sigsoft Workshop On Program Analysis For Software Tools And Engineering
 ACM Transactions On Graphics
 ACM Transactions On Programming Languages And Systems
 ACM Transactions On Software Engineering And Methodology
 ACM-Sigact Symposium On Principles Of Programming Languages
 ACM-Sigplan Conference On Programming Language Design And Implementation
 Ada-Europe International Conference On Reliable Software Technologies
 Annals Of Software Engineering
 Architectural Support For Programming Languages And Operating Systems
 Asia-Pacific Computer Systems Architecture Conference
 Asian Symposium On Programming Languages And Systems
 Aspect-Oriented Software Development
 Australasian Computer Systems Architecture Conference
 Australian Software Engineering Conference
 Australian Workshop On Requirements Engineering (not found in Scopus)
 Automated Software Engineering
 Automated Software Engineering Conference
 Computer
 Computer Aided Geometric Design
 Computer Animation And Virtual Worlds
 Computer Graphics Forum
 Computer Languages
 Computer Languages, Systems And Structures
 Computers & Graphics
 Computers In Entertainment
 Concurrent Systems Engineering Series
 Conference On Agile Software Development
 Conference On The Quality Of Software Architectures
 Crosstalk
 Cutter It Journal
 Empirical Software Engineering
 European Conference On Object-Oriented Programming
 European Conference On Pattern Languages Of Programs
 European Conference On Software Maintenance And Reengineering
 European Software Engineering Conference
 European Software Engineering Conference And The ACM Sigsoft
 European Symposium On Programming
 Fundamental Approaches To Software Engineering
 Graphical Models
 Higher-Order And Symbolic Computation
 IEEE Computer Graphics And Applications
 IEEE International Conference On Software Maintenance And Evolution
 IEEE Multimedia
 IEEE Software
 IEEE Transactions On Software Engineering
 IEEE Transactions On Visualization And Computer Graphics
 IEEE Working Conference On Reverse Engineering
 IEEE/Ifip Working Conference On Software Architecture
 IET Software
 Informatik - Forschung Und Entwicklung
 Information And Software Technology
 Information Visualization
 Innovations In Systems And Software Engineering
 International Computer Software And Applications Conference
 International Conference On Availability, Reliability And Security
 International Conference On Computer Safety, Reliability And Security
 International Conference On Evaluation And Assessment In Software Engineering
 International Conference On Evaluation Of Novel Approaches To Software Engineering
 International Conference On Extreme Programming
 International Conference On Formal Engineering Methods
 International Conference On Functional Programming
 International Conference On Generative Programming And Component Engineering
 International Conference On Quality Software
 International Conference On Reliable Software Technologies
 International Conference On Software And Data Technologies
 International Conference On Software Composition
 International Conference On Software Development (could not verify the existence of above conference)
 International Conference On Software Engineering
 International Conference On Software Engineering And Formal Methods
 International Conference On Software Language Engineering
 International Conference On Software Process
 International Conference On Software Reuse
 International Journal Of Agent Oriented Software Engineering
 International Journal Of Computational Science And Engineering
 International Journal Of Shape Modeling
 International Journal Of Software Engineering & Knowledge Engineering
 International Journal On Software Tools For Technology Transfer
 International Symposium Component-Based Software Engineering
 International Symposium On Code Generation And Optimization
 International Symposium On Empirical Software Engineering And Measurement
 International Symposium On Software Reliability Engineering
 International Symposium On Software Testing And Analysis
 International Symposium On The Foundations Of Software Engineering

International Workshop On Computer-Aided Software Engineering
 International Workshop On Requirements Engineering:
 Foundation For Software Quality
 Jisuanji Xuebao/Chinese Journal Of Computers
 Jisuanji Yanjiu Yu Fazhan/Computer Research And Development
 Joop - Journal Of Object-Oriented Programming
 Journal Of Computer Science And Technology
 Journal Of Functional And Logic Programming
 (outdated name of the Journal of Functional Programming)
 Journal Of Functional Programming
 Journal Of Information Science And Engineering
 Journal Of Integrated Design And Process Science
 Journal Of Object Technology
 Journal Of Software
 Journal Of Software Maintenance And Evolution
 Journal Of Software: Evolution And Process
 Journal Of Systems And Software
 Journal Of Visual Languages And Computing
 Journal Of Visualization And Computer Animation
 Multimedia Tools And Applications
 Pattern Languages Of Programs
 Product Focused Software Process Improvement
 Requirements Engineering
 Ruan Jian Xue Bao/Journal Of Software
 Science In China, Series F: Information Sciences
 Science Of Computer Programming
 Software And Systems Modeling
 Software Process: Improvement And Practice
 Software Quality Journal
 Software Technology And Engineering Practice
 Conference
 Software Testing, Verification And Reliability
 Software: Practice And Experience
 Technology Of Object-Oriented Languages And Systems
 Europe
 The Visual Computer
 Working Conference On Reverse Engineering
 Workshop On Programming Languages And Operating Systems

Appendix C. Rationale for the chosen search strategy

Initially, we attempted a citation search using the key publications¹ of each acceptance behavior theory of interest as a start set. For TAM, for example, we identified five publications that are commonly cited in studies using TAM: Davis' original publications on TAM [29,30,44] and Venkatesh et al.'s extensions of TAM, TAM2 [32] and TAM3 [33].

When searching Scopus (on Feb 14, 2023) for citations of these five publications, we found that two of Davis' original publications [29,30] were not indexed in Scopus at this time. A search for the remaining three provided 35,329 documents, of which 18,260 were categorized as within computer science (search string: "REFEID (2-s2.0-0033872521 OR 2-s2.0-43649087245 OR 2-s2.0-55249087535)"). Searching for citations of the two main versions of the Unified Theory of Acceptance and Use of Technology (UTAUT, [45,46]) provided 22,158 documents, of which 11,597 were categorized as within computer science (search string: "REFEID (2-s2.0-1542382496 OR 2-s2.0-84859868870)"). For the Theory of Planned Behavior (TPB) and the Theory of Reasoned Action (TRA), we could not identify a suitable set of key publications

that are commonly cited in studies using these theories since there is a wide variation of original publications on TPB and TRA. Furthermore, there are many well-cited books and book chapters that are not indexed in common literature databases like Scopus (e.g., [47,48] had 30584 and 9772 citations in Google Scholar, respectively on Feb 14, 2023). One of the key publications that can be searched on Scopus [37] has 46,031 citations alone, of which 16,381 are categorized as within computer science (per Feb 14, 2023).

Taken together, an automated citation search using Scopus was unreliable since many key publications for the acceptance behavior theories of interest were not indexed. A similar (manual) search using Google Scholar was infeasible due to the sheer number of citations (more than 300,000 citations for TAM, UTAUT, and TPB/TRA alone).

We, therefore, used a database search using a search string based on the names of an existing list of relevant acceptance behavior theories (see Section 3.1.1). The result of such a search then needed to be restricted (filtered) to the publications dealing with research in software engineering.

Such a filtering step could, for example, be achieved by adding general search terms like "software engineering" or specific software engineering topics like "software testing", "requirements engineering", or "verification and validation". It is challenging to create a complete list of terms and phrases for all knowledge areas and software development tasks. Furthermore, it is highly likely that research papers do not have these keywords in their title, abstract, and keywords since there is no commonly accepted terminology for the research topics in most software engineering areas. When the keywords are fairly general, a search will result in an unmanageable number of hits. However, it will also miss relevant publications that either use even more general terminology, more specific terminology, or uncommon terminology. Tran et al. [49] illustrate the problems with using the keyword "software" to restrict the search results to software engineering research.

To mitigate the problems described above and identify the use of acceptance behavior theories in software engineering, we used an existing list of relevant acceptance behavior theories (see Section 3.1.1) together with an established list of software engineering venues (see Section 3.1.2) and combined them into the following logical search string:

list of acceptance behavior theories AND

list of peer-reviewed software engineering publication venues

Appendix D. Full search string

Search date: May 9, 2023

Database: Scopus

Number of hits: 197

(TITLE-ABS-KEY ("Big five theory" OR "UTAUT" OR "Expectation confirmation theory" OR "Information-Motivation-Behavioural" OR "Inter-Organizational Relationship theory" OR "Igbaria's model" OR "Unified Theory of Acceptance and Use of Technology" OR "Technology acceptance model" OR "Theory of planned behaviour" OR "Theory of planned behavior" OR "Theory of reasoned action" OR "Innovation diffusion theory" OR "Diffusion of innovation" OR "Motivational model" OR "Motivation model" OR "Model of PC utilization" OR "Social cognitive theory" OR "Technology Integration Model" OR "IS success model" OR "Information Systems success model" OR "Model of Acceptance with Peer Support" OR "Perceived Characteristics of Innovating" OR "Perceived value model" OR "Social capital theory" OR "Social identity theory" OR "Task-technology fit model" OR "Theory of interpersonal behaviour" OR "Technology-organization-environment framework" OR "Stages of change model" OR "Uses and Gratification theory")

AND

(ISSN (0097-8930) OR ISSN (0362-1340) OR ISSN (1523-2867) OR ISSN (0730-0301) OR ISSN (1557-7368) OR ISSN (0164-0925) OR

¹ The key publications for a theory are those publications about a theory that are most commonly cited in papers using said theory.

ISSN (1558-4593) OR ISSN (1049-331x) OR ISSN (1557-7392) OR ISSN (1022-7091) OR ISSN (0928-8910) OR ISSN (1573-7535) OR ISSN (0018-9162) OR ISSN (0167-8396) OR ISSN (1546-4261) OR ISSN (1546-427x) OR ISSN (0167-7055) OR ISSN (1467-8659) OR ISSN (0096-0551) OR ISSN (1477-8424) OR ISSN (0097-8493) OR ISSN (1544-3574) OR ISSN (1383-7575) OR ISSN (1522-7383) OR ISSN (1382-3256) OR ISSN (1573-7616) OR ISSN (1524-0703) OR ISSN (1524-0711) OR ISSN (1388-3690) OR ISSN (1573-0557) OR ISSN (0272-1716) OR ISSN (1070-986x) OR ISSN (0740-7459) OR ISSN (0098-5589) OR ISSN (1077-2626) OR ISSN (1751-8806) OR ISSN (0178-3564) OR ISSN (0949-2925) OR ISSN (0950-5849) OR ISSN (1473-8716) OR ISSN (1473-8724) OR ISSN (1614-5046) OR ISSN (1614-5054) OR ISSN (1746-1375) OR ISSN (1746-1383) OR ISSN (1742-7185) OR ISSN (1742-7193) OR ISSN (0218-6543) OR ISSN (1793-639x) OR ISSN (0218-1940) OR ISSN (1793-6403) OR ISSN (1433-2779) OR ISSN (1433-2787) OR ISSN (0254-4164) OR ISSN (1000-1239) OR ISSN (0896-8438) OR ISSN (1097-1408) OR ISSN (1000-9000) OR ISSN (1860-4749) OR ISSN (1080-5230) OR ISSN (0956-7968) OR ISSN (1469-7653) OR ISSN (1016-2364) OR ISSN (1092-0617) OR ISSN (1660-1769) OR ISSN (1796-217x) OR ISSN (1532-060x) OR ISSN (1532-0618) OR ISSN (2047-7481) OR ISSN (0164-1212) OR ISSN (1045-926x) OR ISSN (1095-8533) OR ISSN (1049-8907) OR ISSN (1099-1778) OR ISSN (1380-7501) OR ISSN (1573-7721) OR ISSN (0947-3602) OR ISSN (1432-010x) OR ISSN (1000-9825) OR ISSN (1009-2757) OR ISSN (1862-2836) OR ISSN (0167-6423) OR ISSN (1619-1366) OR ISSN (1619-1374) OR ISSN (1077-4866) OR ISSN (1099-1670) OR ISSN (0963-9314) OR ISSN (1573-1367) OR ISSN (0960-0833) OR ISSN (1099-1689) OR ISSN (0038-0644) OR ISSN (1097-024x) OR ISSN (0178-2789) OR ISSN (1432-2315)

OR

EXACTSRCTITLE (“Programming Language”) OR EXACTSRCTITLE (“Programming Languages”) OR EXACTSRCTITLE (“Reliable Software Technologies”) OR EXACTSRCTITLE (“Computer Systems Architecture”) OR EXACTSRCTITLE (“Aspect-Oriented Software Development”) OR EXACTSRCTITLE (“Computer Systems Architecture”) OR EXACTSRCTITLE (“Requirements Engineering”) OR EXACTSRCTITLE (“Agile Software Development”) OR EXACTSRCTITLE (“Quality Of Software Architectures”) OR EXACTSRCTITLE (“Crosstalk”) OR EXACTSRCTITLE (“Object-Oriented Programming”) OR EXACTSRCTITLE (“Pattern Languages Of Programs”) OR EXACTSRCTITLE (“Software Maintenance”) OR EXACTSRCTITLE (“Software Engineering”) OR EXACTSRCTITLE (“Programming”) OR EXACTSRCTITLE (“Software Maintenance And Evolution”) OR EXACTSRCTITLE (“Reverse Engineering”) OR EXACTSRCTITLE (“Software Architecture”) OR EXACTSRCTITLE (“Computer Software And Applications”) OR EXACTSRCTITLE (“Availability Reliability And Security”) OR EXACTSRCTITLE (“Computer Safety Reliability And Security”) OR EXACTSRCTITLE (“Extreme Programming”) OR EXACTSRCTITLE (“Formal Engineering Methods”) OR EXACTSRCTITLE (“Functional Programming”) OR EXACTSRCTITLE (“Generative Programming And Component Engineering”) OR EXACTSRCTITLE (“Quality Software”) OR EXACTSRCTITLE (“Reliable Software Technologies”) OR EXACTSRCTITLE (“Software And Data Technologies”) OR EXACTSRCTITLE (“Software Composition”) OR EXACTSRCTITLE (“Software Development”) OR EXACTSRCTITLE (“Software Engineering And Formal Methods”) OR EXACTSRCTITLE (“Software Language Engineering”) OR EXACTSRCTITLE (“Software Process”) OR EXACTSRCTITLE (“Software Reuse”) OR EXACTSRCTITLE (“Code Generation And Optimization”) OR EXACTSRCTITLE (“Software Reliability Engineering”) OR EXACTSRCTITLE (“Software Testing And Analysis”) OR EXACTSRCTITLE (“The Foundations Of Software Engineering”) OR EXACTSRCTITLE (“Computer-Aided Software Engineering”) OR EXACTSRCTITLE (“Requirements Engineering Foundation For Software Quality”) OR EXACTSRCTITLE (“Pattern Languages Of Programs”) OR EXACTSRCTITLE (“Product Focused Software Process Improvement”) OR EXACTSRCTITLE (“Software Technology

And Engineering Practice”) OR EXACTSRCTITLE (“Object-Oriented Languages”) OR EXACTSRCTITLE (“Software Tools And Engineering”) OR EXACTSRCTITLE (“Reverse Engineering”) OR EXACTSRCTITLE (“Software testing”) OR EXACTSRCTITLE (“software development”) OR EXACTSRCTITLE (“software evolution”)

OR

CONF (“Programming Language”) OR CONF (“Programming Languages”) OR CONF (“Reliable Software Technologies”) OR CONF (“Computer Systems Architecture”) OR CONF (“Aspect-Oriented Software Development”) OR CONF (“Computer Systems Architecture”) OR CONF (“Requirements Engineering”) OR CONF (“Agile Software Development”) OR CONF (“Quality Of Software Architectures”) OR CONF (“Crosstalk”) OR CONF (“Object-Oriented Programming”) OR CONF (“Pattern Languages Of Programs”) OR CONF (“Software Maintenance”) OR CONF (“Software Engineering”) OR CONF (“Programming”) OR CONF (“Software Maintenance And Evolution”) OR CONF (“Reverse Engineering”) OR CONF (“Software Architecture”) OR CONF (“Computer Software And Applications”) OR CONF (“Availability Reliability And Security”) OR CONF (“Computer Safety Reliability And Security”) OR CONF (“Extreme Programming”) OR CONF (“Formal Engineering Methods”) OR CONF (“Functional Programming”) OR CONF (“Generative Programming And Component Engineering”) OR CONF (“Quality Software”) OR CONF (“Reliable Software Technologies”) OR CONF (“Software And Data Technologies”) OR CONF (“Software Composition”) OR CONF (“Software Development”) OR CONF (“Software Engineering And Formal Methods”) OR CONF (“Software Language Engineering”) OR CONF (“Software Process”) OR CONF (“Software Reuse”) OR CONF (“Code Generation And Optimization”) OR CONF (“Software Reliability Engineering”) OR CONF (“Software Testing And Analysis”) OR CONF (“The Foundations Of Software Engineering”) OR CONF (“Computer-Aided Software Engineering”) OR CONF (“Requirements Engineering Foundation For Software Quality”) OR CONF (“Pattern Languages Of Programs”) OR CONF (“Product Focused Software Process Improvement”) OR CONF (“Software Technology And Engineering Practice”) OR CONF (“Object-Oriented Languages”) OR CONF (“Software Tools And Engineering”) OR CONF (“Reverse Engineering”) OR CONF (“Software testing”) OR CONF (“software development”) OR CONF (“software evolution”))

AND

(LIMIT-TO (LANGUAGE, “English”))

References

- [1] Nauman Bin Ali, Is effectiveness sufficient to choose an intervention?: Considering resource use in empirical software engineering, in: Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2016, Ciudad Real, Spain, September 8-9, 2016, ACM, 2016, pp. 54:1–54:6.
- [2] Nauman bin Ali, Kai Petersen, Claes Wohlin, A systematic literature review on the industrial use of software process simulation, *J. Syst. Softw.* 97 (2014) 65–85.
- [3] Zoe Kotti, Georgios Gousios, Diomidis Spinellis, Impact of software engineering research in practice: A patent and author survey analysis, *IEEE Trans. Softw. Eng.* 49 (4) (2023) 2020–2038.
- [4] Mahmood Niazi, David Wilson, Didar Zowghi, Critical success factors for software process improvement implementation: An empirical study, *Softw. Process: Improv. Pract.* 11 (2) (2006) 193–211.
- [5] Mahmood Niazi, A comparative study of software process improvement implementation success factors, *J. Softw.: Evol. Process* 27 (9) (2015) 700–722.
- [6] Mathieu Lavallée, Pierre N. Robillard, The impacts of software process improvement on developers: A systematic review, in: Proceedings of the 34th International Conference on Software Engineering, 2012, pp. 113–122.
- [7] Tony Gorschek, Per Garre, Stig Larsson, Claes Wohlin, A model for technology transfer in practice, *IEEE Softw.* 23 (6) (2006) 88–95.
- [8] Dusica Marjan, Arnaud Gotlieb, Industry-academia research collaboration in software engineering: The Certus model, *Inf. Softw. Technol.* 132 (2021) 106473.
- [9] Anna Sandberg, Lars Pareto, Thomas Arts, Agile collaborative research: Action principles for industry-academia collaboration, *IEEE Softw.* 28 (4) (2011) 74–83.
- [10] Klaus Schmid, If you want better empirical research, value your theory: On the importance of strong theories for progress in empirical software engineering research, in: Evaluation and Assessment in Software Engineering, 2021, pp. 359–364.

- [11] D.I.K. Sjöberg, G.R. Bergersen, T. Dybå, Why theory matters, in: *Perspectives on Data Science for Software Engineering*, Elsevier, 2016, pp. 29–33.
- [12] Pontus Johnson, Mathias Ekstedt, Ivar Jacobson, Where's the theory for software engineering? *IEEE Softw.* 29 (5) (2012) 96.
- [13] Martin S Hagger, Linda D Cameron, Kyra Hamilton, Nelli Hankonen, Taru Lintunen, *The Handbook of Behavior Change*, Cambridge University Press, Cambridge, UK, 2020.
- [14] Jürgen Börstler, Nauman bin Ali, Martin Svensson, Kai Petersen, Investigating acceptance behavior in software engineering—Theoretical perspectives, *J. Syst. Softw.* 198 (2023) 111592.
- [15] Kuttamani Tamilmani, Nripendra P. Rana, Yogesh K. Dwivedi, Consumer acceptance and use of information technology: A meta-analytic evaluation of UTAUT2, *Inform. Syst. Front.* 23 (4) (2021) 987–1005.
- [16] Barbara Ann Kitchenham, David Budgen, Pearl Brereton, *Evidence-Based Software Engineering and Systematic Reviews*, CRC Press, 2016.
- [17] Kai Petersen, Sairam Vakkalanka, Ludwik Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, *Inf. Softw. Technol.* 64 (2015) 1–18.
- [18] Mark Turner, Barbara Kitchenham, Pearl Brereton, Stuart Charters, David Budgen, Does the technology acceptance model predict actual use? A systematic literature review, *Inform. Softw. Technol.* 52 (5) (2010) 463–479.
- [19] Tobias Lorey, Paul Ralph, Michael Felderer, Social science theories in software engineering research, in: *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1994–2005.
- [20] Dimitra Karanatsiou, Yihao Li, Elvira-Maria Arvanitou, Nikolaos Misirlis, W. Eric Wong, A bibliometric assessment of software engineering scholars and institutions (2010–2017), *J. Syst. Softw.* 147 (2019) 246–261.
- [21] Eric Archambault, Olivier H. Beauchesne, Julie Caruso, Towards a multilingual, comprehensive and open scientific journal ontology, in: *Proceedings of the 13th International Conference of the International Society for Scientometrics and Informetrics*, 2011, p. 15.
- [22] John P.A. Ioannidis, Kevin W. Boyack, Jeroen Baas, Updated science-wide author databases of standardized citation indicators, *PLoS Biol.* 18 (10) (2020) e3000918.
- [23] Nauman bin Ali, Binish Tanveer, A comparison of citation sources for reference and citation-based search in systematic literature reviews, *e-Inf. Softw. Eng. J.* 16 (1) (2022).
- [24] Aline Valente, Maristela Holanda, Ari Melo Mariano, Richard Furuta, Dilma Da Silva, Analysis of academic databases for literature review in the computer science education field, in: *Proceedings of the 52nd IEEE Frontiers in Education Conference*, 2022, pp. 1–7.
- [25] Byron C. Wallace, Kevin Small, Carla E. Brodley, Joseph Lau, Thomas A. Trikalinos, Deploying an interactive machine learning system in an evidence-based practice center: Abstrackr, in: *Proceedings of the 2nd ACM SIGHIT International Health Informatics Symposium*, 2012, pp. 819–824.
- [26] ISO/IEC/IEEE, *Systems and Software Engineering – Vocabulary*, Standard, International Organization for Standardization, Geneva, Switzerland, 2017, ISO/IEC/IEEE 24765:2017(E).
- [27] Emelie Engström, Margaret-Anne Storey, Per Runeson, Martin Höst, Maria Teresa Baldassarre, How software engineering research aligns with design science: A review, *Empir. Softw. Eng.* 25 (4) (2020) 2630–2660.
- [28] Dale L. Goodhue, Ronald L. Thompson, Task-technology fit and individual performance, *MIS Q.* 19 (2) (1995) 213–236.
- [29] Fred D. Davis, A Technology Acceptance Model for Empirically Testing New End-User Information Systems: Theory and Results (Ph.D. thesis), Massachusetts Institute of Technology, Boston, MA, USA, 1985.
- [30] Fred D. Davis, Richard P. Bagozzi, Paul R. Warshaw, User acceptance of computer technology: A comparison of two theoretical models, *Manage. Sci.* 35 (8) (1989) 982–1003.
- [31] Martin Fishbein, Icek Ajzen, *Belief, Attitude, Intention and Behavior: An Introduction to Theory and Research*, Addison-Wesley, Reading, MA, USA, 1975.
- [32] Viswanath Venkatesh, Fred D. Davis, A theoretical extension of the technology acceptance model: Four longitudinal field studies, *Manage. Sci.* 46 (2) (2000) 186–204.
- [33] Viswanath Venkatesh, Hillol Bala, Technology acceptance model 3 and a research agenda on interventions, *Decis. Sci.* 39 (2) (2008) 273–315.
- [34] Daniel L. Moody, The method evaluation model: A theoretical model for validating information systems design methods, in: *Proceedings of the 11th European Conference on Information Systems*, 2003, pp. 1–17.
- [35] Nikola Marangunic, Andrina Granic, Technology acceptance model: A literature review from 1986 to 2013, *Univ. Access Inform. Soc.* 14 (1) (2015) 81–95.
- [36] Everett M. Rogers, *Diffusion of Innovations*, third ed., The Free Press, New York, NY, USA, 1983, First edition 1962.
- [37] Icek Ajzen, The theory of planned behavior, *Organ. Behav. Hum. Decis. Processes* 50 (2) (1991) 179–211.
- [38] Mark T. Dishaw, Diane M. Strong, Extending the technology acceptance model with task-technology fit constructs, *Inform. Manag.* 36 (1) (1999) 9–21.
- [39] Albert Bandura, Self-efficacy: Toward a unifying theory of behavioral change, *Psychol. Rev.* 84 (2) (1977) 191–215.
- [40] Albert Bandura, Cultivate self-efficacy for personal and organizational effectiveness, in: Edwin A. Locke (Ed.), *Handbook of Principles of Organizational Behavior*, John Wiley & Sons, 2012, pp. 179–200.
- [41] Jiming Wu, Hongwei Du, Toward a better understanding of behavioral intention and system usage constructs, *Eur. J. Inform. Syst.* 21 (6) (2012) 680–698.
- [42] Verena Tiefenbeck, Anna Kupfer, Liliane Ableitner, Samuel Schöb, Thorsten Staake, The uncertain path from good intentions to actual behavior: A field study on mobile app usage, in: *Proceedings of the Twenty-Sixth DIGIT Workshop*, 2016, p. 17.
- [43] Apostolos Ampatzoglou, Stamatia Bibi, Paris Avgeriou, Marijn Verbeek, Alexander Chatzigeorgiou, Identifying, categorizing and mitigating threats to validity in software engineering secondary studies, *Inf. Softw. Technol.* 106 (2019) 201–230.
- [44] Fred D. Davis, Perceived usefulness, perceived ease of use, and user acceptance of information technology, *MIS Q.* 13 (3) (1989) 319–340.
- [45] Viswanath Venkatesh, Michael G. Morris, Gordon B. Davis, Fred D. Davis, User acceptance of information technology: Toward a unified view, *MIS Q.* 27 (3) (2003) 425–478.
- [46] Viswanath Venkatesh, James Y.L. Thong, Xin Xu, Consumer acceptance and use of information technology: Extending the unified theory of acceptance and use of technology, *MIS Q.* 36 (1) (2012) 157–178.
- [47] Icek Ajzen, From intentions to actions: A theory of planned behavior, in: Julius Kuhl, Jürgen Beckmann (Eds.), *Action Control*, Springer, 1985, pp. 11–39.
- [48] Martin Fishbein, Icek Ajzen, *Predicting and Changing Behavior: The Reasoned Action Approach*, Psychology Press, New York, NY, USA, 2011.
- [49] Huynh Khanh Vi Tran, Jürgen Börstler, Nauman Bin Ali, Michael Unterkalmsteiner, How good are my search strings? Reflections on using an existing review as a quasi-gold standard, *e-Inf. Softw. Eng. J.* 16 (1) (2022) 220103.

Included primary studies

- [P1] Temitope Elizabeth Abioye, Oluwasefunmi Tale Arogundade, Sanjay Misra, Adio T. Akinwale, Olusola John Adeniran, Toward ontology-based risk management framework for software projects: An empirical study, *J. Softw.: Evol. Process* 32 (12) (2020).
- [P2] Mark T. Dishaw, Diane M. Strong, Assessing software maintenance tool utilization using task-technology fit and fitness-for-use models, *J. Softw. Mainten. Evol.* 10 (3) (1998) 151–179.
- [P3] Scott A. Bailey, Susan Elliott Sim, Applying models of technology adoption to software tools and methods: An empirical study, in: *Proceedings of the 18th International Conference on Software Engineering and Knowledge Engineering*, 2006, pp. 200–205.
- [P4] Jack Downey, Muhammad Ali Babar, On identifying the skills needed for software architects, in: *Proceedings - International Conference on Software Engineering*, 2008, pp. 1–5.
- [P5] Gregor Polančič, Marjan Heričko, Ivan Rozman, An empirical examination of application frameworks success based on technology acceptance model, *J. Syst. Softw.* 83 (4) (2010) 574–584.
- [P6] Medha Umarji, Carolyn Seaman, Predicting acceptance of software process improvement, in: *Proceedings of the 2005 Workshop on Human and Social Factors of Software Engineering*, HSSE 2005, 2005.
- [P7] Hidekuni Tsukamoto, Yasuhiro Takemura, Hideo Nagumo, Kenichi Matsumoto, Analysis of the motivation of learners in the in-house training of programming in Japanese ICT industries, in: 2011 24th IEEE-CS Conference on Software Engineering Education and Training, CSEE and T 2011 - Proceedings, 2011, pp. 121–128.
- [P8] Parastoo Mohagheghi, Wasif Gilani, Alin Stefanescu, Miguel A. Fernandez, An empirical study of the state of the practice and acceptance of model-driven engineering in four industrial cases, *Empir. Softw. Eng.* 18 (1) (2013) 89–116.
- [P9] Jörg Zettel, Methodology support in CASE tools and its impact on individual acceptance and use: A controlled experiment, *Empir. Softw. Eng.* 10 (3) (2005) 367–394.
- [P10] Leandro Ferreira D'Avila, Jorge Luis Victória Barbosa, Kleinner Silva Farias de Oliveira, SW-Context: A model to improve developers' situational awareness, *IET Softw.* 14 (5) (2020) 535–543.
- [P11] Felipe Ramos, Antônio Pedro, Marcos Cesar, Alexandre Costa, Mirko Perkusich, Hyggo Almeida, Angelo Perkusich, Evaluating software developers' acceptance of a tool for supporting agile non-functional requirement elicitation, in: *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*, vol. 2019-July, 2019, pp. 26–31.
- [P12] Gina C. Green, Alan R. Hevner, Rosann Webb Collins, The impacts of quality and productivity perceptions on the use of software process improvement innovations, *Inf. Softw. Technol.* 47 (8) (2005) 543–553.
- [P13] Fleming Woo, Romas Mikusauskas, Dean Bartlett, Rob Law, Is OO the systems development technology for your organization? in: *Proceedings - Fourth International Conference on Software Engineering Research, Management and Applications, SERA 2006*, 2006, pp. 354–361.
- [P14] Gregor Polančič, Romana Vajde Horvat, Ivan Rozman, Improving object-oriented frameworks by considering the characteristics of constituent elements, *J. Inform. Sci. Eng.* 25 (4) (2009) 1067–1085.

- [P15] Rakesh Rana, Miroslaw Staron, Jörgen Hansson, Martin Nilsson, Wilhelm Meding, A framework for adoption of machine learning in industry for software defect prediction, in: ICSE/EA 2014 - Proceedings of the 9th International Conference on Software Engineering and Applications, 2014, pp. 383–392.
- [P16] Ian Owens, Dave Sammon, John McAvoy, A sense-making approach to agile method adoption, in: ICSE/EA 2008 - Proceedings of the 3rd International Conference on Software and Data Technologies, vol. ISDM, 2008, pp. 292–295, Issue: ABF/-.
- [P17] Cynthia K. Riemenschneider, Bill C. Hardgrave, Fred D. Davis, Explaining software developer acceptance of methodologies: A comparison of five theoretical models, *Trans. Softw. Eng.* 28 (12) (2002) 1135–1145.
- [P18] Gregor Polančič, Gregor Jošt, The impact of the representatives of three types of process modeling tools on modeler's perceptions and performance, *J. Softw.: Evol. Process* 28 (1) (2016) 27–56.
- [P19] Rachel Vital, Glauca Melo, Toacy Oliveira, Paulo Alencar, Don Cowan, AgileCritPath: Identifying critical tasks in agile environments, in: Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE, vol. 2019-July, 2019, pp. 20–25.
- [P20] Amantia Pano, Daniel Grazieriotin, Pekka Abrahamsson, Factors and actors leading to the adoption of a JavaScript framework, *Empir. Softw. Eng.* 23 (6) (2018) 3503–3534, eprint: 1605.04303.
- [P21] Kuo Chung Chang, Jiun Hung Wong, Yuzhu Li, Yao Chin Lin, Houn Gee Chen, External social capital and information systems development team flexibility, *Inf. Softw. Technol.* 53 (6) (2011) 592–600.
- [P22] Luis Rivero, Auri Vincenzi, José Carlos Maldonado, Tayana Conte, Evaluating software engineers' acceptance of a technique and tool for web usability inspection, in: Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE, vol. 2015-January, 2015, pp. 140–145.
- [P23] Irit Hadar, Tomer Hasson, Oshrat Ayalon, Eran Toch, Michael Birnhack, Sofia Sherman, Arod Balissa, Privacy by designers: Software developers' privacy mindset, *Empir. Softw. Eng.* 23 (1) (2018) 259–289.
- [P24] Martin Dick, The benefits of object technology in a greenfields site, in: "Technology of Object-Oriented Languages and Systems, 1999, pp. 286–294, Issue: TOOL 32.
- [P25] Yue Yu, Gang Yin, Huaimin Wang, Tao Wang, Exploring the patterns of social behavior in Github, in: 1st International Workshop on Crowd-Based Software Development Methods and Technologies, CrowdSoft 2014 - Proceedings, 2014, pp. 31–36.
- [P26] Soren Lauesen, Mohammad A. Kuhail, Task descriptions versus use cases, *Requir. Eng.* 17 (1) (2012) 3–18.
- [P27] M. Ciolkowski, J. Heidrich, J. Munch, F. Simon, M. Radicke, Evaluating software project control centers in industrial environments, in: First International Symposium on Empirical Software Engineering and Measurement, ESEM 2007, IEEE, Madrid, Spain, 2007, pp. 314–323.
- [P28] Igor Steinmacher, Tayana Uchoa Conte, Christoph Treude, Marco Aurélio Gerosa, Overcoming open source project entry barriers with a portal for newcomers, in: Proceedings - International Conference on Software Engineering, vol. 14-22-May-2016, 2016, pp. 273–284.
- [P29] Silvia Abrahão, Geert Poels, A family of experiments to evaluate a functional size measurement procedure for Web applications, *J. Syst. Softw.* 82 (2) (2009) 253–269.
- [P30] Katsiaryna Labunets, Fabio Massacci, Federica Paci, Le Minh Sang Tran, An experimental comparison of two risk-based security methods, in: International Symposium on Empirical Software Engineering and Measurement, 2013, pp. 163–172.
- [P31] Muhamad Ali Babar, Dietmar Winkler, Stefan Biffl, Evaluating the usefulness and ease of use of a groupware tool for the software architecture evaluation process, in: Proceedings - 1st International Symposium on Empirical Software Engineering and Measurement, ESEM 2007, 2007, pp. 430–439.
- [P32] Mark T. Dishaw, Diane M. Strong, Supporting software maintenance with software engineering tools: A Computed task-technology fit analysis, *J. Syst. Softw.* 44 (2) (1998) 107–120.
- [P33] Diogo Matheus Costa, Eldānae Nogueira Teixeira, Cláudia Maria Lima Werner, Evaluating the usefulness and ease of use of a software process line tool, in: 23rd Iberoamerican Conference on Software Engineering, CIBSE 2020, 2020.
- [P34] Müller Miranda, Renato Ferreira, Fernando Figueira Filho, Cleidson R.B. De Souza, Leif Singer, An exploratory study of the adoption of mobile development platforms by software engineers, in: 1st International Conference on Mobile Software Engineering and Systems, MOBILESoft 2014 - Proceedings, 2014, pp. 50–53.
- [P35] Mali Senapathi, Ananth Srinivasan, Understanding post-adoptive agile usage: An exploratory cross-case analysis, *J. Syst. Softw.* 85 (6) (2012) 1255–1268.
- [P36] Andreas L. Opdahl, Guttorm Sindre, Experimental comparison of attack trees and misuse cases for security threat identification, *Inf. Softw. Technol.* 51 (5) (2009) 916–932.
- [P37] Carol Passos, Manoel Mendonça, Daniela S. Cruzes, The role of organizational culture in software development practices: A cross-case analysis of four software companies, in: Proceedings - 28th Brazilian Symposium on Software Engineering, SBES 2014, 2014, pp. 121–130.
- [P38] Carol Passos, Daniela S. Cruzes, Manoel Mendonça, Beliefs underlying teams intention and practice: An application of the theory of planned behavior, in: CIBSE 2013: 16th Ibero-American Conference on Software Engineering - Memorias Del 10th Workshop Latinoamericano Ingenieria de Software Experimental, ESELAW 2013, 2013, pp. 103–116.
- [P39] Maja Pušnik, Katja Harej Pulko, Marjan Heričko, Matjaž B. Jurič, Boštjan Šumak, Investigation of developer's perceptions in xml schema development using textual and visual tool types, *Int. J. Softw. Eng. Knowl. Eng.* 24 (3) (2014) 445–463.
- [P40] Sardar Muhammad Sulaman, Armin Beer, Michael Felderer, Martin Höst, Comparison of the FMEA and STPA safety analysis methods—a case study, *Softw. Qual. J.* 27 (1) (2019) 349–387.
- [P41] Carol Passos, Daniela S. Cruzes, Manoel Mendonça, Applying theory of reasoned action in the context of software development practices: Insights into team intention and behavior, in: Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering, 2013, pp. 2–11.
- [P42] Verusca Rocha, Sávio Freire, Manoel Mendonça, Rodrigo Spínola, Evaluating a conceptual framework for supporting technical debt management in testing activities—a feasibility study, in: Proceedings of the 7th Brazilian Symposium on Systematic and Automated Software Testing, 2022, pp. 69–78.
- [P43] Leandro Flores Da Silva, Edson Oliveira Jr., Evaluating usefulness, ease of use and usability of an UML-based software product line tool, in: Proceedings of the 34th Brazilian Symposium on Software Engineering, 2020, pp. 798–807.
- [P44] Miroslaw Ochodek, Krystian Koronowski, Adam Matysiak, Piotr Miklosik, Sylwia Kopczyńska, Sketching use-case scenarios based on use-case goals and patterns, in: Software Engineering: Challenges and Solutions: Results of the 18th KKIO Software Engineering Conference, 2016, pp. 17–30.
- [P45] Luciano A. Garcia, Edson Oliveira Jr., Marcelo Morandini, Tailoring the Scrum framework for software development: Literature mapping and feature-based support, *Inf. Softw. Technol.* 146 (2022) 106814.
- [P46] Adriana Lopes, Igor Steinmacher, Tayana Conte, UML acceptance: Analyzing the students' perception of UML diagrams, in: Proceedings of the 33rd Brazilian Symposium on Software Engineering, 2019, pp. 264–272.
- [P47] Nur Atiqah Sia Abdullah, Rusli Abdullah, Mohd Hasan Selamat, Azmi Jaafar, User acceptance for extended function point analysis in software security costing, in: Proceedings of the Second International Conference on Software Engineering and Computer Systems, vol. 2, 2011, pp. 346–360.